

Support automation that knows when to answer and when to hand over

A retrieval-augmented answering system for CloudServe Solutions. It sends a reply when the documentation supports one, and otherwise hands the ticket to a person with the search results and the draft attached.

Author	Alok Kulkarni
Project	Forward Deployed AI Engineering Capstone
Client	CloudServe Solutions (fictional)
Submission	27 September 2026
Graded evidence	final_val_paid_80_20260927: 80 validation tickets, gpt-4o-mini drafting, gpt-4o guardrails, cache off
Free-model evidence	final_dev_local_50_20260927: 50 development tickets, llama3.1-8b-ctx8k drafting, qwen2.5-7b-ctx8k guardrails, \$0.00
Code version	47e0bc1

Contents

1	Executive summary	3
2	The problem	4
3	Discovery findings	6
4	Requirements	9
5	Architecture and design	11
6	Implementation	14
7	Evaluation	16
8	Governance and risk	20
9	The requirements revision	22
10	Conclusions	23
A	Full results tables	24
B	Intent precision, recall and confusion matrix	30
C	Fairness audit	32
D	Decision index	33
E	Prompt register	34
F	Dataset profile	35
G	Workbooks and artefacts	36
H	Code listings	37
I	Declaration and AI-tool use	38

SECTION 1

Executive summary

CloudServe Solutions asked for a chatbot. The problem behind the request was a support queue of more than five hundred tickets a week handled by six agents, where most tickets asked questions CloudServe's own help articles already answered and nobody on the team used those articles. This project built a system that reads a ticket, searches the 29 articles, drafts a reply, checks the draft with six guardrails that block rather than warn, and only then decides whether to send it, hand the ticket to a person with the working attached, or block the draft.

The graded run put all 80 validation tickets through one unattended command on 27 September 2026 with the response cache off, so every figure comes from live model calls. Table 1.1 gives the headline results. A second run took 50 development tickets through free local models at no cost, which shows the pipeline completes without a paid provider.

Measure	Target	Graded run (80 validation tickets)	Status
First-contact resolution	60%	56.2% correctly resolved (76.2% auto-sent)	NOT MET
Highest resolution possible		60.0% (48 of 80 tickets should be answered)	
Escalation rate	30% or lower	23.8%	MET
Mean time to first reply	under 5 min	5.6 s	MET
Intent accuracy		96.2% (77 of 80)	
Latency p95	under 3 s	6.7 s	NOT MET
Private data in sent replies	0	0	MET
Decisions logged	all	80 of 80, reconciled	MET

Table 1.1: Headline results of the graded run. Appendix A has all twelve measures.

Only 48 of the 80 tickets should be answered automatically, so 60 per cent is both the ceiling and the target. The system sent correct replies to 45 of those 48; human agents resolved 46.2% of the same tickets at first contact according to their recorded history. Escalation fell to 23.8% from a 58 per cent baseline, and no private data reached a customer.

The most important caveat: the system also sends confident replies to questions the documentation cannot answer. 16 replies went to tickets the labels say should have been held, 15 of them unanswerable from the articles, and nothing the pipeline computes before sending separates them from the correct replies. Four Framework measures (satisfaction, hallucination rate, sentence-level citation accuracy and the manual private-data review) need human reviewers the project did not have, and are reported as not measured.

SECTION 2

The problem

What CloudServe asked for

CloudServe Solutions is a cloud infrastructure vendor of about 150 employees and 200 corporate customers. Support is the part of the operation that has broken first. Marcus Adeyemi, the head of support, described a team of six receiving more than five hundred tickets a week (EV-M1). The service agreement promises a first reply within two hours; replies take eight to twelve. First-contact resolution is 42 per cent against the 65 per cent industry benchmark he is measured on, every escalated ticket costs about four times what a resolved one does, satisfaction has fallen to 3.2 out of 5, and two experienced agents left in the last quarter citing workload.

The request that came out of that meeting was for a chatbot. The word carries an assumption about the shape of the answer: a conversational front end that takes the first contact away from the humans. Building exactly that would have met the request and missed the problem. A chat interface decides how a reply is delivered. It says nothing about where the answer comes from, whether it is right, or what happens when no answer exists, and those are the questions the queue turns on.

What each person could see

Five people were interviewed. Table 2.1 gives the line from each that did the most work in the design. Each of them described their own part of the problem accurately, and none of them described the whole.

Person	Role	What they said
Marcus Adeyemi	Head of Support	"I would rather it said nothing than said something wrong."
Sofia Restrepo	Tier 1 agent	Seven in ten tickets are patterns she has seen that month; searching the docs is painful, so most agents do not.
Daniel Okonkwo	Tier 2 engineer	"About half of what reaches me is something tier one could have resolved if they had been confident, or if they had found the right page."
Ines Varga	Technical writer	"Internally, I do not think the support team uses [the docs] at all."
Ravi Menon	Customer	"If I know a machine drafted it I will verify first. Hiding that would be the thing that annoys me."

Table 2.1: The five interviews. Quotes from workbooks/discovery_notes.md; brackets mark a word added for sense; Sofia's line is paraphrased from two answers.

Put together, the interviews describe a different problem from the one requested. The answers to most tickets already exist in 29 reviewed help articles, and the people who need them cannot find them. Agents rebuild answers from memory or from private files of old replies, some of them two years out of date (EV-D2). Tier one under-answers when unsure. Escalations reach tier two as a forwarded ticket with no note of what was tried, so Daniel asks customers questions they already answered (EV-D1). Meanwhile the customer waits eight to twelve hours for an answer that sits in an article.

The gap between the request and the problem

The internal search matches titles and exact words, while customers describe symptoms in their own words. Ines's example was a customer writing "my deployment keeps dying" against an article titled "resolving container health check failures", with no word in common (EV-I2). That makes the core job semantic retrieval and a judgement about whether the retrieved article answers the ticket, not conversation. It also sets the failure that matters most. CloudServe's customers are engineers, and Marcus expects a confidently wrong reply to be screenshotted and public within the hour (EV-M3).

The success measures follow from the same interviews. Marcus cares more about resolution rising than about reply time falling (EV-M2). Sofia wants a draft with the relevant article, not a finished auto-answer (EV-S5). Daniel wants escalations to arrive with the ticket, what tier one thought it was, the articles found

and the point of doubt (EV-D3). Ravi will accept an automated reply if it is honest about being automated, says where its answer came from and is followed by a person when needed (EV-R2). Three topics must never be answered automatically: security, billing disputes and data location (EV-D4). Table 2.2 shows how these became measures.

Stakeholder need	Evidence	How the system measures or enforces it
Resolve more at first contact	EV-M1, EV-M2	First-contact resolution against labels, with its ceiling (7.1)
Never be confidently wrong	EV-M3	Six blocking guardrails; wrong-send count (7.4)
Escalations with context	EV-D1, EV-D3	EscalationBundle on every hold (FR-11)
Say where the answer came from	EV-R2	Citations on every reply (FR-13)
Never auto-answer certain topics	EV-D4	Fixed intent policy list (D-07, FR-10)
Explainable for the autumn compliance review	EV-M5	Decision log reconciled every run (FR-20, NFR-09)
No worse for any customer group	EV-S4, EV-R1, EV-M4	Fairness audit on every run (Section 8, Appendix C)

Table 2.2: From stakeholder need to measure.

Marcus still describes the outcome as a chatbot, because that is what he could picture. Ines and Daniel describe it in terms of the queue. The PRD reconciles the two by taking Marcus's numeric targets as tier-one measures, Ines's answerability observation as a constraint on retrieval, and Daniel's request as the escalation contract. The FastAPI endpoint that exposes the pipeline exists for the demonstration and for Prometheus; no requirement asks for a conversational interface.

SECTION 3

Discovery findings

Discovery used two sources: the five interview transcripts and the 500 development tickets. The transcripts were read three times, for what people said, for where they contradicted each other, and for what they did not seem to know. The interviews left four open questions, and the ticket data was used to settle each one (Table 3.1). The supporting tables are in Appendix F.

Question	Who said what	What the ticket data shows
Q1. What share can the docs answer?	Sofia about 70%; Marcus did not know	357 of 500 development tickets (71.4%) are answerable from the articles.
Q2. Are escalations hard cases or avoidable?	Marcus assumed hard; Daniel said about half avoidable	138 of the 281 tickets humans escalated (49.1%) were answerable from the articles.
Q3. Bad content or bad search?	Ines heard content; Sofia meant search	Search. Meaning-based retrieval finds a correct article in the top three for 91.0% of answerable development tickets (D-02), and 98.1% of answerable validation tickets after tuning.
Q4. Is service worse for some groups?	Sofia saw a fluency gap; Ravi a tier gap	The history shows no fluency gap (first-contact resolution 43.2% fluent, 45.8% non-fluent). Enterprise customers fared worst (37.3%).

Table 3.1: The four open questions and how the data settled them.

Finding one: a documented answer exists for most tickets

Sofia estimated seven in ten tickets were familiar patterns (EV-S3), and the labels agree: 71.4% of the development tickets are marked answerable from the articles. This is the largest lever in the system. It made retrieval quality the technical measure that matters most, and it set the ceiling the evaluation reports: on the validation set 60.0% of tickets should be answered automatically, on the development set 62.2%. The finding also says what the articles do not cover. Feature requests, roadmap questions and new incidents have no article, and inventing an answer for them is the wrong behaviour (EV-I3).

Figure 3.1 breaks the answerable share down by intent. Two intents are never answerable, feature requests and unclear requests, because no article can answer them. Every other intent sits between 54% and 96%. Answerability is therefore a property of the individual ticket, not of its intent, and no rule keyed on intent alone can decide it. That observation returns in Section 7.4, where most wrong sends are unanswerable tickets with an ordinary intent.

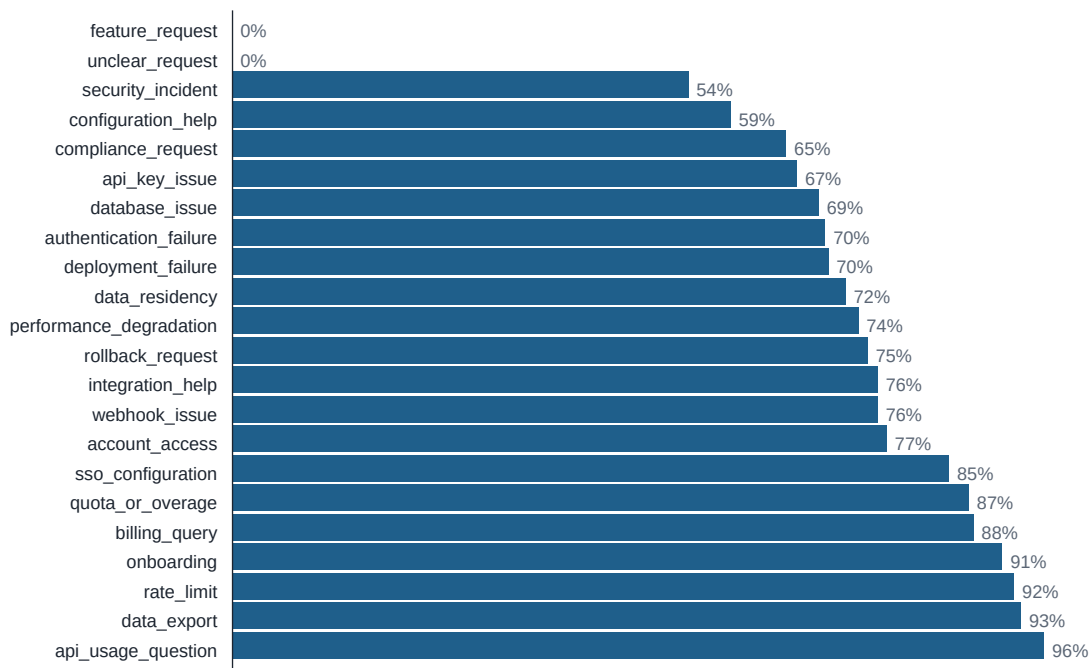


Figure 3.1: Share of development tickets answerable from the 29 articles, by labelled intent (n=500).

Finding two: half of today's escalations were avoidable

Marcus assumed escalations were the hard cases, since that is what tier two exists for. Daniel said about half were tickets tier one could have closed with more confidence or the right page. The history settles it in Daniel's favour: 138 of the 281 development tickets humans escalated were answerable from the articles. The queue is a sorting failure more than a capacity failure, which is why the system routes on answerability and hands over the working rather than simply answering faster.

Finding three: precision matters more than coverage

Marcus's line, "I would rather it said nothing than said something wrong" (EV-M3), is precision preferred over recall, stated twice in the same interview. It is the evidence tag that does the most design work. It is why the confidence floor exists (FR-19), why four intents never auto-respond (D-07), why every guardrail blocks, and why the evaluation reports wrong sends separately from wrong holds. Ravi's conditions (EV-R2) point the same way: a reply should say where it came from, so every reply carries citations.

Finding four: escalations arrive without context

Daniel described an escalated ticket as a raw ticket with a note attached. Tier one has already searched, formed a view of the intent and sometimes drafted a reply, and none of it reaches him. His request became FR-11: every hold carries the retrieved passages, the classifier's alternatives, the draft (flagged if a guardrail blocked it) and the rule that stopped the send. It is also why routing runs last; before generation and validation, the bundle would be empty.

Finding five: the problem is finding the articles, not their content

Ines had heard from Sofia that the content was bad; Sofia meant the search. The data sides with the search reading. Meaning-based retrieval with all-MiniLM-L6-v2 found a correct article in the top three for 91.0 per cent of the 357 answerable development tickets, within three points of a TF-IDF baseline and better than it on non-fluent English (D-02). Prepending each article's title and category to its chunks raised that to 95.2 per cent (D-04), and on the validation set it is 98.1 per cent. The articles contain the answers; the internal title search could not reach them from the words customers use.

The ticket mix shaped the ingest requirements. The development set has 212 email, 155 chat, 78 docs_comment, 55 forum tickets, and 155 of the 155 chat tickets have no subject at all, which is why FR-02 requires ingest to accept an empty subject and why retrieval reads the subject and body together (D-09).

Finding six: the service gap is not where people thought

Sofia believed non-fluent customers got the worst service (EV-S4) and Ravi believed business-tier customers waited longer than enterprise (EV-R1). The recorded history supports neither. Non-fluent customers were resolved at first contact 45.8% of the time against 43.2% for fluent customers, with CSAT 3.04 against 2.95. Enterprise customers had the lowest first-contact resolution (37.3%) and the longest median resolution time (369 minutes against 141 for business). The fairness risk was kept (R-04) because a retrieval system can create a fluency gap the human process did not have, and Section 8 reports what the audit found.

Finding seven: the labels disagree with themselves

Fifty groups of identical subject and body carry contradictory routing or answerability labels across the labelled sets, 144 tickets in all. 21 of the 80 validation tickets are affected on an exact match, and 39 once case, punctuation and spacing are ignored. Two pairs sit inside the validation set (VAL-0012 and VAL-0034; VAL-0060 and VAL-0061), where no system can be right on both. This is a finding about the measurement rather than the system. The labels were not edited, because changing ground truth to match model output would make every figure unfalsifiable; Section 7.4 reports how much the disagreement affects the results.

What discovery ruled out

A holding reply was ruled out because Sofia said customers already ignore the automated acknowledgement, and the PRD places it out of scope. A multi-turn conversational interface was ruled out for the reason the chatbot framing was: the evidence points at answerability and triage, not conversation.

SECTION 4

Requirements

The PRD (`workbooks/Stage_2_PRD_Template.docx`) holds 24 functional requirements (FR-25 was removed on 2 September) and nine non-functional ones, NFR-01 split into three parts. Each names the evidence that produced it and the acceptance criterion it serves. Table 4.1 shows where each component's requirements live.

Component	Functional	Non-functional	Anchoring decisions
Ingest	FR-01 to FR-03	NFR-03 reliability	FR-01 v2 strips labels
Classify	FR-04, FR-05	NFR-05 fairness, NFR-07 calibration	PR-CLASSIFY-01
Retrieve	FR-06 to FR-08	NFR-01a context relevance	D-02, D-02a, D-04, D-09
Generate	FR-13 to FR-15	NFR-01b groundedness, NFR-01c answer relevance	D-06, D-06a
Validate	FR-16 to FR-19		D-11, D-12, D-14, D-16
Route	FR-09 to FR-12	NFR-04 explainability, NFR-06 determinism	D-05b, D-07, D-14, D-15
Decision log	FR-20	NFR-09 auditability	D-03, D-03a, D-03b
Harness and tests	FR-21 to FR-24	NFR-02 latency, NFR-08 cost	D-08, D-13, D-17

Table 4.1: Where each component's requirements live.

How the requirements followed from the findings

Finding one produced the retrieval requirements (FR-06 to FR-08) and NFR-01a, which asks that an answerable ticket gets at least one passage from an expected article. Finding three produced FR-19, FR-10 with its intent policy list, and NFR-01b, which asks that every factual claim in a reply has a supporting span. Finding four produced FR-11 and the decision to route last. The explainability and auditability requirements (NFR-04, FR-20, NFR-09) come from Marcus's autumn compliance review (EV-M5). Finding six produced the rule that every results table carries a limitations block.

Acceptance criteria

The Build Specification sets twelve acceptance criteria. Table 4.2 gives the status of each on the submitted code and the graded run.

	Criterion	Status and evidence
A1	Runs from a clean checkout via the README	Met. Checked on 27 September from a fresh clone on macOS with Python 3.11: install, verify, index, harness, tests and API all ran. The README now states Python 3.11 and gives PowerShell steps.
A2	All four channels ingested	Met. The graded run held 22 chat, 16 docs_comment, 31 email, 11 forum tickets.
A3	Numeric confidence in [0,1] on every ticket	Met on all 80 rows.
A4	Retrieval returns identifiable passages	Met. Every row records the doc_ids retrieved.
A5	Routing is deterministic	Met. route() is a pure function; tests/test_route.py.
A6	Citations resolve to retrieved passages	Met: 0 of 80 rows cite an article that was not retrieved.
A7	At least one guardrail can block	Met. 5 drafts blocked on the graded run.
A8	Every decision logged and reconciled	Met. 80 of 80.
A9	Full set unattended in one command	Met. One harness call per run.
A10	Metrics report without manual work	Met. metrics_report.json and the results table are written by the run.
A11	Degrades gracefully	Met. On the local run one of 50 tickets hit a model fault and escalated with a reason instead of failing the run.
A12	Tests pass with one command	Met. python -m pytest tests/ -v: 712 passed, 6 skipped.

Table 4.2: Acceptance criteria.

Non-functional targets that were missed

Three non-functional targets are not met, and they are stated here rather than left for the reader to find. NFR-02 asks for a p95 of 4.0 seconds; the graded run's is 6.7 s. NFR-07 asks for calibration within 5 points in the 0.80 to 0.90 band; the observed gap is +9.9. NFR-08 asks that a full validation run fits the provider's free tier; the graded run used a paid provider at \$0.91, and the free local stack is the configuration that meets it.

Two requirements that changed during the build

FR-10 v1 escalated tickets marked `must_not_auto_respond` in the labels. That lasted one day: labels are evaluation data, and FR-01 v2 keeps them off the Ticket the router sees. v2 uses a fixed list of four intents plus unknown (D-07), which reproduces the label at precision and recall 1.000 on all 580 labelled tickets. FR-19 v1 blocked a reply on low classifier confidence, which names the wrong outcome because no draft exists at that point. v2 escalates instead (D-14); on the run before the fix, 17 of 33 blocks had the confidence floor as their only failing check.

Traceability can be checked in one command. `grep -r "FR-11" src/ tests/ prompts/` lands on the bundle builder in `src/route.py` and its test in `tests/test_route.py`, and FR-11 cites EV-D1 and EV-D3 in the PRD.

SECTION 5

Architecture and design

The system is a router over six stages, shown in Figure 5.1: ingest, classify, retrieve, generate, validate and route. The routing decision runs last, which is not what the word router usually suggests. Every ticket is classified, searched, drafted and checked, and only then does route() read what those stages produced and choose one of three outcomes. Two of its rules read results that exist nowhere earlier: whether a blocking guardrail failed, and whether the generator declined to answer.

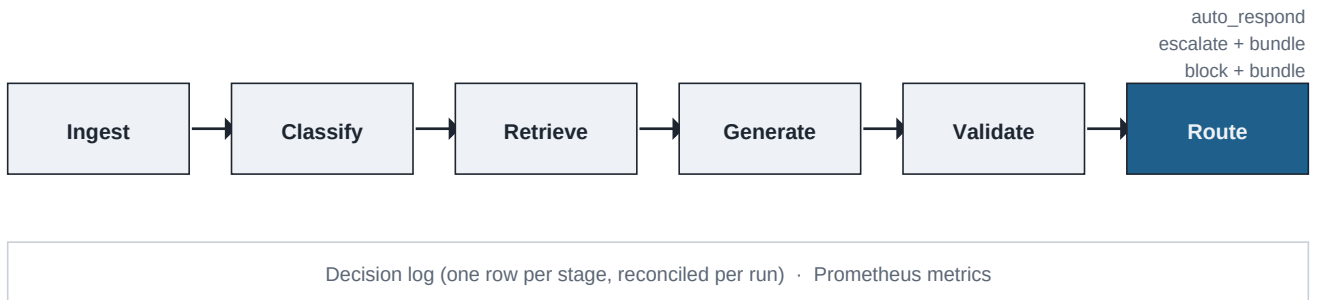


Figure 5.1: The six-stage pipeline. Routing reads the results of the five stages before it. The decision log and metrics touch every stage.

The first draft of the architecture put routing fourth, before generation. The code never worked that way and could not have: routing before validation leaves the block outcome unreachable and turns the guardrails into advice. The documented order was corrected on 20 September. Table 5.1 describes what each stage does.

Stage	What it does
Ingest	Normalises email, chat, docs comments and forum posts into one Ticket, and strips labels and history so production code cannot see answers. Handles the empty subject every chat ticket has.
Classify	Returns one of 22 intents or unknown, an urgency, a confidence and up to two alternatives. It never sees tier, region, name or fluency, so tier-aware policy lives in the router where it can be inspected.
Retrieve	Searches 59 chunks of the 29 articles (800 characters with 120 of overlap, title and category prepended) with all-MiniLM-L6-v2 in cosine space. Below the 0.25 floor it returns nothing rather than something irrelevant.
Generate	Writes a JSON reply with citations and an unknown flag. The ticket is fenced as data. A structural self-check requires every citation to be a retrieved article and allows one retry.
Validate	Six blocking guardrails, at most four running at once: private data, grounding, instruction integrity, tone and scope, answer relevance and the confidence floor.
Route	A pure function of what the stages produced, with no clock, randomness or model call. Eight ordered rules, first match wins (Table 5.2).

Table 5.1: What each stage does.

Rule	Condition	Outcome
0	Kill switch set (D-15)	escalate every ticket
1	A classification or generation decision was not logged (D-03b)	escalate
2	A blocking guardrail failed on the draft (confidence floor excluded, D-14)	block, draft kept for the reviewer
2b	The ticket text looks like an instruction to the system (D-16)	escalate, recorded
3	Intent is on the never-auto-respond list (D-07)	escalate
4	Retrieval returned nothing	escalate
5	The generator declined to answer	escalate
6	Classifier confidence below 0.85	escalate
	None of the above	auto_respond

Table 5.2: Routing rules in precedence order, from `src/route.py` (listing in Appendix H).

The order of Table 5.2 is part of the contract, because FR-12 requires the reason on each decision to name the cause, and a ticket often trips more than one rule. On an escalation or block the router returns the `EscalationBundle`: the passages, the classifier's top three intents, the draft with a flag if a check found a fault, and the rule that stopped the send. That is the context Daniel asked for.

Table 5.4 shows a real bundle from the graded run. VAL-0045 asked whether to cancel a two-million-record export still queued after a weekend. The draft added advice the article does not give, so grounding blocked it, and the reviewer receives the article the system found, its second guess at the intent, the withheld draft and the reason, instead of a bare forwarded ticket.

Field	Value
Ticket	I started an export over the weekend and it is still showing as queued. It covers about two million records. Should I cancel and try again?
Decision	block (trigger guardrail_blocked)
Passages	DOC-DATA-001 Exporting account data and scheduled extracts (best score 0.58); DOC-API-002 Pagination and handling large result sets (best score 0.44); DOC-DEPLOY-002 Rolling back a failed release (best score 0.39); DOC-SEC-003 Audit logging and compliance evidence (best score 0.39)
Alternatives	api_usage_question 0.75
Draft	Exports covering more than a million records commonly take several minutes to process, as they are queued and processed asynchronously. It may be worth waiting for the export to complete rather than canceling and trying again immediately. If the export does not complete, you can check the exports page for job status. If you decide to cancel, you can always initiate a new export later. Please note that the export refl...
Draft blocked	yes
Why it was held	block: 1 blocking guardrail(s) did not pass (grounding); the drafted reply is withheld and a human must review it — grounding: 2 unsupported claim(s): 'It may be worth waiting for the export to complete rather than canceling and try': not present in cited passage; 'If you decide to cancel, you can always initiate a new export later.': not present in cited passage

Table 5.4: The `EscalationBundle` the reviewer receives for VAL-0045, graded run.

The guardrails

All six guardrails block; none warns, which is acceptance criterion A7 and Marcus's precision preference. The private-data check runs a regular-expression pass and a model pass, and blocks only on detections shaped like a real value (D-11), after the local judge had flagged the words "API keys" as a key. Grounding checks every factual claim against the passages. Instruction integrity blocks a reply in which the ticket redirected the system. Tone and scope blocks commitments about refunds, timelines or the roadmap. Answer relevance compares the reply with the question and deliberately does not see the passages, because a grounded reply can still answer a question nobody asked. A check that fails to run

blocks the draft: it fails safe. Table 5.5 lists the six with the prompt and requirement behind each. Three of the checks can share one model call (PR-GUARDRAIL-DRAFT-01), built to stop re-sending the same instructions to each check; their separate prompts remain the specification and the fallback.

Guardrail	Blocks when	How it checks	Traces to
Private data	The reply contains an email, key, phone number, account number or another customer's name	Regular expressions plus PR-GUARDRAIL-PII-01	FR-16, R-02
Grounding	A factual claim has no supporting span in the passages	PR-GUARDRAIL-GROUNDING-01	FR-17, NFR-01b
Instruction integrity	The reply shows the ticket redirected the system, or cites an article outside the passages	Pure Python, no model call	FR-18, R-03
Tone and scope	The reply commits to a refund, a timeline or a roadmap item	PR-GUARDRAIL-TONESCOPE-01	EV-D4, R-04
Answer relevance	The reply does not address the question asked	PR-GUARDRAIL-RELEVANCE-01, without the passages	FR-14, R-01
Confidence floor	Classifier confidence is below 0.85	Threshold, read at rule 6	FR-19, D-14

Table 5.5: The six guardrails.

Alternatives considered

Each non-obvious choice is recorded as an ADR with the options, the choice, the reason and what would change it; Appendix D lists all of them. Table 5.3 gives six where an alternative was measured and rejected.

ADR	Rejected	Chosen	Why
D-02	TF-IDF; bge-small-en	all-MiniLM-L6-v2	91.0% top-3 hit on 357 development tickets, better on non-fluent English
D-03	PostgreSQL; JSON lines	SQLite	Nothing to provision; reconciliation is one query
D-06	ReAct; full Self-RAG	Router with a structural critic	Search is close to solved on this corpus; the value is after it
D-10	Retrieval-score answerability gate	No gate	Six features reach AUC 0.64; the gate held 0 of 14 wrong sends
D-12	Guardrails in sequence	Concurrent, four workers	Guardrail stage 10.1 s to 3.1 s, identical verdicts
D-17	File-order queue	Urgent tickets first	An interrupted run spends its calls on urgent tickets; the rating's accuracy is in 7.2

Table 5.3: Alternatives measured and rejected.

Layers

Three layers sit under the stages. The application layer is the evaluation harness (evaluation/harness.py) and a FastAPI service with POST /ticket, GET /healthz and GET /metrics; the harness is the graded path and the service runs the same six calls in the same order. The domain layer is the six stage modules. The persistence layer is the SQLite decision log, the Chroma vector store and the Prometheus registry. Swapping the model provider or the vector store does not touch the domain code, which is why moving from a failing free model to paid diagnostics and then to a local stack was a configuration change rather than a rewrite.

SECTION 6

Implementation

How it was built

The build followed the Stage 4 sprint plan and the Build Specification's checkpoints. In week two, ingest ran end to end on one ticket per channel on day one; retrieval returned traceable passages on day two; classification and routing wrote decisions with reasons on day three; generation and the first blocking guardrail worked on day four; and on day five the first unattended 80-ticket run wrote its metrics report. Week three went on the evaluation, on ADRs D-08 to D-16, which closed issues week two had surfaced, and on the submission. Table 6.1 lists the modules.

Module (src/)	Purpose
<code>ingest.py</code>	Normalises a raw ticket into a Ticket; strips labels and history
<code>classify.py</code>	Intent, urgency and confidence (PR-CLASSIFY-01)
<code>retrieve.py</code> , <code>index_docs.py</code>	Search over the indexed corpus; one-time indexer
<code>generate.py</code>	Grounded reply with citations and the structural self-check
<code>guardrails.py</code>	The six blocking guardrails
<code>route.py</code>	The eight routing rules and the EscalationBundle
<code>injection.py</code>	Input-side detection of prompt-injection attempts (D-16)
<code>logging_store.py</code>	SQLite decision log with per-run reconciliation
<code>model_call.py</code> , <code>rate_limit.py</code>	Caller-side deadline on each call (D-13); pacing only after the provider refuses
<code>model_cache.py</code> , <code>usage.py</code>	On-disk reply cache (D-08); tokens and cost per call
<code>prompt_loader.py</code>	Loads versioned prompts by ID
<code>api.py</code> , <code>metrics.py</code>	FastAPI service; Prometheus metrics
<code>config.py</code> , <code>schema.py</code> , <code>logging_config.py</code>	Settings from the environment; shared models; structured logging

Table 6.1: Source modules.

Testing

The suite has 36 files under `tests/` and runs with one command, `python -m pytest tests/ -v`; on the submitted code 712 tests pass and 6 are skipped. There is one file per module for the components in isolation, end-to-end tests of the harness and the API with stubbed providers, an explicit blocking case for each guardrail, and governance checks such as the fairness audit, the calibration sheet and a credential scan that fails CI if a key pattern appears in the source. Tests never call a real provider.

What was difficult

Free-tier rate limits were the first problem. The Setup Guide's free model moved to paid pricing in week two, and both recorded free-tier runs lost most calls to outages or HTTP 429. The Build Specification calls rate limits a design problem, so the response was a design: an on-disk reply cache keyed on the request (D-08), a retry budget raised from two to five with pacing only after a refusal, and a local Ollama stack that closes the free-model gate (D-01a).

Long-tail latency was the second. A 20 September run hung for more than seven minutes on a healthy endpoint. The socket timeout measures the gap between bytes, and a provider streaming keep-alive comments resets it indefinitely. Every model call now runs under a caller-side deadline of 180 seconds (D-13); without it one hung call takes the unattended run with it.

Guardrail latency was the third. Six checks in turn took about ten seconds a ticket on hosted models. Running them through a pool of four workers cut the stage from 10.1 to 3.1 seconds with identical

verdicts on 20 replayed drafts (D-12). The confidence floor was the fourth: it had returned block at rule 2 alongside real draft failures, and 17 of 33 blocks on one run had it as the only failing check. It now escalates at rule 6 (D-14).

Changes since the 20 September report

The harness now classifies every ticket first and works the high-urgency ones first (D-17); order cannot change a completed run's figures, only what an interrupted run spent its calls on. The results table gained per-class precision and recall with the intent confusion matrix, a repeat-contacts row with a baseline taken from each ticket's history, and an availability row, all three Framework requirements the earlier report did not meet. Every file the harness reads or writes now names UTF-8: on Windows the default encoding is cp1252, which cannot write the table's less-than-or-equal signs, so the run finished without its results table. A regression test reproduces the Windows default on any platform. The README states Python 3.11, because the pinned numpy and pandas have no wheels for 3.12 or later, and gives PowerShell equivalents.

What I would restructure

The generator and the guardrails should overlap. Today the generator writes a whole draft and then the checks read it. Running the checks against the draft as it streams is the largest latency saving available, since grounding and private-data checks do not need the last sentence to judge the first. The decision log should be wiped at the start of each run rather than scoped by run id; I kept the persistent file because one file of history helps the demonstration and the scoped query is simple. And the FastAPI service is thin: serving real traffic would need request queuing, per-tenant rate limits and a webhook back to the ticketing system, none of which the PRD asks for.

SECTION 7

Evaluation

Method

Run harness-20260927T101501Z-10995e3f covered all 80 validation tickets on 2026-09-27. It used gpt-4o-mini to draft and gpt-4o to run the guardrails, with the confidence threshold at 0.85, all guardrails on and the model cache off, so all 304 calls were live (650,349 prompt and 25,828 completion tokens). It cost \$0.91 and took 7.0 minutes. Results are in `evaluation/results/final_val_paid_80_20260927`.

Run harness-20260927T101501Z-32ab3564 covered the first 50 development tickets on 2026-09-27. It used llama3.1-8b-ctx8k to draft and qwen2.5-7b-ctx8k to run the guardrails, with the confidence threshold at 0.85, all guardrails on and the model cache off, so all 198 calls were live (429,130 prompt and 16,449 completion tokens). It cost \$0.00 and took 44.5 minutes. Results are in `evaluation/results/final_dev_local_50_20260927`.

The development tickets were used to build the system, so the local run shows that the free models complete the pipeline; it is not an independent test of accuracy. The validation run is the graded figure. Runs against the hidden evaluation set: zero. The harness takes an input path so the assessor can point it at that file. Correctness throughout is agreement with the dataset's labels. Table 7.0 maps each Framework requirement to where this section reports it.

Framework requirement	Status
Resolution with escalation rate; reply time mean and median	Measured (7.1)
Satisfaction proxy: rubric, sample, scorers	Not measured: no human scorers
Repeat contacts: baseline, target halved	Baseline measured from history; the system's own rate needs production data (7.1)
Intent precision and recall per class; confusion matrix	Measured (7.2, Appendix B)
Hallucination rate: 50 replies, two assessors	Not measured: one assessor
Citation accuracy: each citation against its sentence	Not measured to method; article-level match (7.2)
Latency p95 end to end	Measured, with stage split (7.2)
Availability, including provider failure	Measured (7.2)
Private data: automated scan and manual sample	Scan measured; manual sample not done
Quality across groups; decision logging; calibration	Measured (7.3, Appendix C)
Date, ticket count, version, hidden-set runs	Stated above

Table 7.0: Framework coverage.

7.1 Business outcomes

Table 7.1 gives the tier-one measures for the graded run.

Measure	Baseline	Target	Achieved	Confidence
First contact resolution	42%	60%	56.2% correctly resolved (76.2% auto-sent) (NOT MET)	95% CI 45-67% (n=80)
Mean time to first reply	8 to 12 hrs	< 5 min	5.6 s (MET)	n=61 auto-sent tickets; median 5.5 s
Satisfaction proxy	3.2 / 5	4.0	Not measured (NOT MEASURED)	No scored sample on this run
Escalation rate	58%	≤ 30%	23.8% (MET)	95% CI 16-34% (n=80)
Repeat contacts	21.2% (history, n=80)	Halved	Not measured (NOT MEASURED)	An offline run cannot observe a customer returning

Table 7.1: Tier-one outcomes, graded run.

First-contact resolution counts only correct sends, and the labels cap it. 48 of the 80 tickets should be answered automatically. The other 32 are 27 the articles cannot answer and 5 that policy sends to a person (three security incidents, one compliance request and one performance ticket). The ceiling is therefore 60.0%, exactly the target, and the development set leaves the same margin (62.2%). The run answered 45 of the 48 and held 3, so those 3 tickets are the whole shortfall. Human agents resolved 46.2% of the same tickets at first contact.

Escalation is 23.8% against a 58 per cent baseline and meets the 30 per cent target. The system answers more than the humans did, and the cost of that shows up as wrong sends (7.4) rather than as queue depth. Mean time to a first automated reply is 5.6 s, measured from reading the ticket to the reply being ready; queueing and delivery are excluded, and held tickets get their first reply from the human queue.

For repeat contacts the Dataset Guide defines `history.*` as what CloudServe achieves today. 21.2% of these tickets came back under human handling, so halving it means 10.6%. An offline run cannot see a customer return, so the system's own rate is not measured; the 16 wrong sends are the replies most likely to come back. Satisfaction is not measured either. The Framework asks for a stated sample of replies scored by people against a rubric, and no such scoring was done. For reference, the tickets' history records a mean CSAT of 2.60 under human handling.

7.2 Technical performance

Intent accuracy is 96.2% (77 of 80). 95.7% mean over the 21 classes with a precision denominator; 18 of 21 at $\geq 85\%$. The target is per class, so one class below 85 per cent fails it. Precision is below 85 per cent for `compliance_request` (50% of 2 predictions); `account_access` (80% of 5 predictions); `api_usage_question` (80% of 5 predictions), and recall for `rate_limit` (0% of 1 tickets); `authentication_failure` (67% of 3 tickets); `data_residency` (83% of 6 tickets). Most classes have fewer than five validation tickets, so one ticket moves a class by 20 points or more. Appendix B has every class and the confusion matrix.

Urgency is the weaker prediction. Against `labels.urgency` it is right 36.2% of the time, below the 43.8% that always answering medium would score, and it rated 8 of the 25 high-urgency tickets as high. The labels are hard to predict from anything: on the development set only security incidents and rollbacks have a dominant urgency, and tier, channel, region and fluency each move the share of high tickets by no more than 14 points. D-17 orders the queue by this rating, so its benefit is not demonstrated on this data.

Retrieval finds an expected article in the top three for 98.1% of the 53 answerable tickets; recall at one, three and five passages is 79.2%, 88.7% and 90.6%. Retrieval is close to solved on this corpus, as the D-06 pilot predicted.

Citation accuracy is not measured to the Framework's method, which checks each citation against the sentence it supports and needs a reviewer. 29 of the 61 sent replies put their only citation after the last sentence, so for those the check would have to be made against the reply as a whole. The article-level match is reported instead: the cited ARTICLE was one the labels expect in 94.3% of citations over 53 drafts (recall 82.1%), a check of the right source, not of support. Hallucination rate is not measured either: the method needs at least 50 replies assessed by two people independently, and the project had one assessor. The grounding guardrail does check every claim in every draft and blocks on any unsupported one.

Latency p95 is 6.7 s against the 3-second target and the PRD's 4.0 (n=80 live tickets; max 7.2 s). Table 7.2 splits it by stage: generation and the guardrails account for most of it, and a ticket makes 3.8 model calls on average.

Stage	p50 s	p95 s	mean s
classification	1.302	1.565	1.312
retrieval	0.102	0.252	0.127
generation	1.782	2.65	1.815
guardrails	2.085	2.652	1.965

Table 7.2: Where the time goes, graded run, all tickets live.

Availability is 100.0%: every ticket had every stage work (95% CI 95-100% (n=80)). Eighty tickets cannot demonstrate 99.5 per cent, because the upper bound on the failure rate at this size is about 5 per cent. Behaviour under provider failure is tested rather than observed on this run: a ticket whose model call fails is escalated with a reason and never dropped. On the local run one of 50 tickets hit a model fault (DEV-0037, a draft cut off at the local model's length limit) and still received a decision, and the 19 September local run completed 80 of 80 against a failing free provider.

7.3 Governance conditions

Table 7.3 gives the four tier-three conditions and Table 7.4 the calibration bands behind the last of them.

Condition	Requirement	Result	Status
Private data in outbound text	Zero occurrences.	0 (Automated scan of all 61 sent replies; manual sample review: not done)	MET
Quality across customer groups	Under five percentage points of variation.	17 pts (language_fluency: non_fluent vs fluent) (breach not significant: non_fluent n=19 against fluent n=61; 2 segments under n=10 excluded)	NOT MEASURED
Decision logging	Complete coverage.	80 of 80 tickets logged; reconciles=True	MET
Confidence calibration	Stated confidence within five points of observed accuracy.	largest gap +9.9 pts in band 0.8-0.9 (stated 0.85, observed 0.95, n=59); 0.80-0.90 band observed 0.95 (n=59, Framework expects about 0.85)	NOT MET

Table 7.3: Tier-three conditions, graded run.

Confidence band	n	Stated	Observed	Gap (points)
0.3-0.4	3	0.33	1.00	+66.7 (n too small)
0.5-0.6	2	0.50	1.00	+50.0 (n too small)
0.6-0.7	1	0.60	1.00	+40.0 (n too small)
0.8-0.9	59	0.85	0.95	+9.9
0.9-1.0	15	0.95	1.00	+5.0

Table 7.4: Calibration. Bands with no tickets are omitted.

The operational band, 0.8 to 0.9, holds 59 tickets and is under-confident: stated 0.85, observed 0.95. That errs in the safe direction but is outside the 5-point condition. The low bands hold too few tickets to judge. Private data and decision logging, the two hard binaries, both hold.

7.4 Where the wrong decisions come from

Table 7.5 attributes each of the 19 wrong decisions to the earliest stage that explains it. Answerability dominates: the system sent replies to tickets the articles cannot answer.

Stage and reason	Tickets
answerability:sent_unanswerable	15
guardrail:grounding:verdict	3
other:sent_against_label	1

Table 7.5: Wrong decisions by cause, graded run.

Contested labels explain little of this. 8 of the 16 wrong sends and 1 of the 3 wrong holds are tickets whose text recurs elsewhere with the opposite label. Setting contested tickets aside on both sides, correct and incorrect alike, send precision is 34 of 44 (77.3%) on an exact match and 18 of 26 (69.2%) on the normalised match, against 73.8% overall. On the 23 September development run (dev_local_80_20260923) 15 of 18 wrong sends sat on contested text, so how much the labels matter depends on the sample. Some near-duplicates escape both matches: VAL-0024 and VAL-0049 describe the same read-only key in almost the same words and carry opposite labels.

Read one at a time, the wrong sends look like sound replies from the right article. The labels call for escalation mostly where the customer had already tried the documented step. VAL-0025 reports raising a health-check timeout twice; VAL-0056 reports the same in almost the same words and is labelled for an automatic reply. The system sent both the same answer, which suggests a port mismatch the customer had not tried. A rule that escalates whenever a customer mentions a prior attempt would also hold five correct sends on this run, and on the development set such tickets are labelled for escalation less often, 23 per cent against 40. Neither a confidence threshold nor a retrieval score separates the wrong sends: they were classified at the same two confidence levels as the correct sends, and D-10 found the retrieval-feature gate held none of them.

The 3 wrong holds (VAL-0008, VAL-0029, VAL-0045) were drafts the grounding guardrail blocked for a claim not in the cited passages. On the paid model these are usually detail the model added, such as advice the article does not give, which is the block working as designed.

7.5 Free-model run

Table 7.6 sets the local run beside the graded one. The ticket sets differ, so the comparison shows behaviour, not a controlled difference in accuracy.

Measure	Local, 50 development tickets	Paid, 80 validation tickets
First-contact resolution	60.0% correctly resolved (80.0% auto-sent)	56.2% correctly resolved (76.2% auto-sent)
Highest resolution possible	64.0%	60.0%
Escalation rate	20.0%	23.8%
Wrong sends / wrong holds	10 / 2	16 / 3
Send precision	75.0%	73.8%
Intent accuracy	96.0%	96.2%
Mean time to first reply	52.2 s	5.6 s
Latency p95	77.9 s	6.7 s
Private data in sent replies	0	0
Tickets with a model fault	1 of 50	0 of 80
Cost	\$0.00	\$0.91

Table 7.6: Local and paid runs side by side.

The figures above should be treated with caution because

one run of 80 tickets moves any rate by 1.25 points per ticket, and the 95 per cent intervals in Appendix A are 20 points wide or more. Correctness is agreement with labels, and 21 of the 80 validation tickets (39 on a normalised match) carry text that is labelled the other way elsewhere, with two such pairs inside the validation set. Four measures need human reviewers and are not measured: satisfaction, hallucination, sentence-level citation accuracy and the manual private-data review. Most intent classes have fewer than five validation tickets. The local run uses development tickets the system was built against. The hidden evaluation set has not been run, and the confidence threshold was swept on the validation set, which is where the figures are most exposed.

SECTION 8

Governance and risk

The Governance Framework asks for a risk register with real mitigations, decision logging for every automated decision, a fairness audit, an incident procedure someone unfamiliar with the system could follow at two in the morning, and a declaration. This section covers each; the full workbook is submitted alongside.

Risk register

Eight risks are registered. Table 8.1 gives the six whose mitigation is built into the design rather than promised. The other two, stale documentation (R-05) and latency under load (R-07), are recorded in the workbook and covered by out-of-scope items in the PRD.

ID	Risk	Mitigation in the design	Evidence, 27 September
R-01	Confident, incorrect answers	Confidence floor, blocking grounding check, four never-auto-respond intents	16 wrong sends, 15 of them on unanswerable tickets (7.4)
R-02	Private data in a reply	Private-data guardrail on every draft, value-shaped patterns (D-11)	0 in 61 sent replies, automated scan
R-03	Ticket text treated as instruction	Ticket fenced as data; input-side check escalates (D-16)	Six-attack probe: none succeeded
R-04	Worse answers for some groups	Fairness audit on every run	non-fluent decision correctness 63% against 80% fluent, not significant at n=19
R-06	Provider unavailable	Reply cache, caller-side deadline, local stack	availability 100.0%; faults escalate, never drop
R-08	Cannot stop it quickly	Kill switch read on every routing call (D-15)	Tested: test_kill_switch_forces_escalate

Table 8.1: Risks with a mitigation built into the design.

Fairness audit

`evaluation/fairness_audit.py` runs on every graded run. It segments decision correctness, resolution rate, unsafe-send rate and send precision by language fluency, region, tier, channel and ticket length, and applies the PRD's NFR-05 limits with 95 per cent intervals. On the graded run no limit is breached, and 7 checks cannot be settled either way at these segment sizes. The largest gap is language fluency: non-fluent tickets were routed correctly 63% of the time (n=19) against 80% for fluent ones, a gap of 17 points, down from 25 in the 15 September audit. That is above the Framework's 5 points and the PRD's 15, but the two segments' intervals overlap, so the gap is not established at n=19. The human history showed no fluency gap (Section 3), so if this gap is real the system created it. Retrieval depends on the customer's phrasing matching the articles, and a query-rewriting pass at retrieval time is the targeted fix. Appendix C has every segment.

Decision logging

Every stage writes a row to `storage/decisions.db` before the next stage runs, following the Governance Framework's minimum record: ticket, stage, model, prediction, confidence, alternatives, sources, threshold, action, reason, guardrail results, prompt version and requirement ids, plus a run id so each run reconciles on its own. At the end of a run the harness counts logged tickets against processed tickets. The graded run logged 80 of 80 and reconciles. A failed log write is recorded and the ticket escalated rather than sent (D-03b), because an unlogged reply cannot be explained to the compliance review.

Incident procedure

Table 8.2 is the incident procedure from the governance workbook. The kill switch in step two is what makes containment a matter of seconds: exporting `KILL_SWITCH=1` makes the router escalate every

ticket from the next one on, with no restart and no deploy, and it is covered by a test.

Step	What to do	Who	How long
1. Detect	A customer or agent reports a bad reply, or a run's figures move	Support duty manager	Same day for a report; within one run for a metric
2. Contain	Set KILL_SWITCH=1 in the running process's environment (D-15)	Whoever holds the switch	Seconds
3. Assess	Pull the ticket's decision-log row and results entry	Author; a second reader for private data	Under an hour per ticket
4. Notify	Head of Support first, with the decision id and the reply as sent	Author, then Head of Support	Same day; at once for private data
5. Remediate	Fix at the layer named: prompt, guardrail rule, threshold or article	Author; Ines for an article	One to three days
6. Review	Record cause, fix and evidence in the revision log	Author	Within a week

Table 8.2: Incident procedure, from docs/Governance_Framework.docx.

Declaration

Table 8.3 gives the four statements the Governance Framework asks for.

Statement	Position
This system must never	send an unreviewed reply that contains another customer's data, an unsupported factual claim, or a commitment about refunds, timelines or the roadmap.
The mechanism that enforces it	six blocking guardrails on every draft, and a router that turns a block into a hand-over to a person.
The most likely way it could still cause harm	a confident reply to a ticket the documentation does not answer; 15 of the 16 wrong sends were of this kind.
We would not deploy without	running the hidden set, wiring reviewer verdicts back as the measure, and a human-scored sample for satisfaction and hallucination.

Table 8.3: Governance declaration.

SECTION 9

The requirements revision

The Stage 5 revision log (workbooks/Stage_5_PRD_Revision_Log.docx) records nine substantive changes between PRD v1.0 and v2.0. Four of them reshaped the design rather than tightening wording.

The first stopped FR-10 reading a label the router cannot see. v1 escalated any ticket marked `must_not_auto_respond`; the first code review of `route.py` found the router never receives labels, because using them at inference is leakage. v2 uses the D-07 intent list, which reproduces the label exactly on all 580 labelled tickets. The second made FR-19 escalate instead of block, after an audit of the pre-fix run showed 17 of 33 blocks were floor-only and 7 of those were policy tickets rule 3 should have taken. The third gave FR-11 a blocked-draft case, prompted by Daniel's comment in the mid-build walk-through: "show me what got caught, not just that it did." The fourth gave R-08 its own kill switch, because the old mechanism, a threshold of 1.01, needed a restart, shared a constant with threshold tuning, read in the log as a misconfiguration and could not answer the checklist question of how it was tested.

Assumptions that turned out wrong

A retrieval score could stand in for answerability. It cannot: at the 0.25 floor only 14 of 143 unanswerable development tickets return nothing, and six retrieval features reach AUC 0.64 (D-10). An LLM judge could stand in for human evaluation. It cannot on this task: against one human on 50 drafts its Spearman correlation was 0.35 against a 0.70 bar, so its scores are excluded from every reported figure. The free model would stay free for three weeks; it did not. And, found in this revision, the classifier's urgency rating would be good enough to order the queue by; on the labels it scores below a constant.

What I decided not to change

The confidence threshold stays at 0.85. The sweep across 0.50 to 0.95 found no threshold reaching the 0.95 precision Marcus asked for, and the wrong sends in this run sit at the same confidence levels as the correct ones, so moving the threshold trades coverage without separating them.

What the revision taught me

What I most misunderstood in v1 was the routing decision. I drew it as a branch in the middle of the pipeline, with generation and the guardrails hanging off `auto_respond`. Routing has to run last because two of its rules read verdicts from later stages, and that is why the escalation bundle can carry a blocked draft. The discovery work that would have caught it earlier was a walk-through of one escalated ticket with Daniel before any code was written: his request for what was searched, what it was thought to be and any draft implies routing after generation. I read it as a request about data shape rather than control flow. If I started again I would build the fairness audit before the harness, so that every run reported the fluency gap from the first day.

SECTION 10

Conclusions

The system runs from a clean checkout under one command, processes the evaluation set unattended, and writes a metrics report, a results table and a reconciling decision log. On the graded run it answered 76% of tickets, met the escalation target at 23.8%, sent no private data, and resolved 45 of the 48 tickets that could be resolved: 56.2% against the humans' 46.2% on the same tickets.

Two things stand between it and deployment. It sends confident replies to tickets the documentation cannot answer (15 on this run), and nothing it computes before sending tells those apart from correct sends. Its p95 latency is 6.7 s against 3.

What I would do next

The first piece of work is a model-judged sufficiency check before sending, asking whether the passage resolves this customer's situation, including what they say they already tried. It can be measured by replaying the stored drafts before it is built, and D-10 already sets the bar: hold most wrong sends while losing under 5 per cent of correct ones. The second is the reviewer feedback loop the architecture designs: five verdict values written back from the review screen (sent as drafted, edited, rewritten, wrong article, no article), so production outcomes replace dataset labels as the measure; it collects data for a person to act on and must not retune anything automatically, or two runs over the same input would disagree. The third is latency, by running the guardrails against the draft as it streams. The fourth is a human-scored sample of 50 replies by two reviewers, which closes satisfaction, hallucination and citation accuracy in one exercise.

What remains uncertain

Whether 0.95 precision is reachable on this corpus at all. No threshold reaches it, and none of the routes to it (a sufficiency check, query rewriting, a better answerability model) has been shown to get there. Whether the hidden set moves the numbers: it carries whatever labels it carries, and the threshold was swept on the validation set. Whether the fluency gap in the audit is real: at $n=19$ the interval cannot say.

What I would tell CloudServe

That the system does what the queue needs: it answers what it can defend, holds what it cannot, and shows the reviewer its working every time. It resolves nearly every ticket that can be resolved and leaves the queue shorter. Its weakness is replying to questions the articles only appear to cover, and the next version should measure that with reviewer feedback before trusting any dataset label further.

APPENDIX A**Full results table: graded run**

From `evaluation/results/final_val_paid_80_20260927/results_table.md`. Run `harness-20260927T101501Z-10995e3f`, 80 tickets, drafting `gpt-4o-mini`, guardrails `gpt-4o`, threshold 0.85, cache off, hidden-set runs 0. Table A.1 lists every measure.

Measure	Baseline	Target	Achieved	Confidence	Notes
First contact resolution	42%	60%	56.2% correctly resolved (76.2% auto-sent) (NOT MET)	95% CI 45-67% (n=80)	16 of 61 sent replies went to tickets the labels say must be held, so they are not counted as resolved: a wrong answer returns as a repeat contact. Correctness is agreement with labels.expected_route, not a check of reply quality.
Mean time to first reply	8 to 12 hrs	< 5 min	5.6 s (MET)	n=61 auto-sent tickets; median 5.5 s	Harness time from ticket read to reply ready; excludes queueing and delivery. The 19 escalated or blocked tickets get no automated reply (holding reply is out of scope, PRD Table 6), so their first reply comes from the human queue.
Satisfaction proxy	3.2 / 5	4.0	Not measured (NOT MEASURED)	No scored sample on this run	The Framework's method is human review of a stated sample of responses against a rubric, reporting sample size, rubric and scorers. Not done for this run. Nearest evidence: B-18, where one person scored 50 drafts from b21_openai_80_20260914 on the judge's RAG rubric (mean answer relevance 4.86/5) — a different rubric, a different run and a single scorer, so not a satisfaction proxy.
Escalation rate	58%	≤ 30%	23.8% (MET)	95% CI 16-34% (n=80)	14 escalated by routing and 5 blocked by a guardrail; 0 tickets were blocked because a check could not run (fail-safe), not because it found a fault.
Classification precision	—	85%	95.7% mean over the 21 classes with a precision denominator; 18 of 21 at ≥85% (NOT MET)	Precision n (predictions made per class): median 3; 13 classes were predicted fewer than 5 times and 1 never, so per-class figures are indicative only	Weakest: compliance_request 50% (n=2); account_access 80% (n=5); api_usage_question 80% (n=5). 1 class(es) never predicted, so their precision is undefined rather than 0% and is excluded from the mean: rate_limit. Overall intent accuracy 96.2%. The target is per class, so it is met only if every class reaches 85%.
Hallucination rate	—	≤ 5%	Not measured to method (NOT MEASURED)	—	The Framework requires at least fifty responses assessed independently by two people, with their agreement reported. This project had one human assessor. Nearest evidence (B-18, drafts from b21_openai_80_20260914): one human found an unsupported claim in 1 of 50 drafts; the LLM judge in 5 of 50; the judge failed calibration (pooled Spearman 0.3492 against 0.70).
Citation accuracy	—	95%	Not measured to method (NOT MEASURED)	—	The Framework checks each citation against the sentence it is attached to. Sentence-level support was not assessed. Nearest evidence: the cited ARTICLE was one the labels expect in 94.3% of citations over 53 drafts (recall 82.1%) — a check of the right source, not of support.
Latency p95	—	< 3 s	6.7 s (NOT MET)	n=80 live tickets; max 7.2 s	End to end per ticket, including retrieval and every model call, run in sequence. Models: gpt-4o-mini drafting, gpt-4o guardrails.
Private data occurrences	—	0	0 (MET)	Automated scan of all 61 sent replies; manual sample review: not done	The scan uses the PII guardrail's own patterns (email, API key, phone, account number), independently of the judge's verdict. The guardrail blocked 0 drafts for private data before sending.

Measure	Baseline	Target	Achieved	Confidence	Notes
Cross-group variation	—	< 5 pts	17 pts (language_fluency: non_fluent vs fluent) (NOT MEASURED)	breach not significant: non_fluent n=19 against fluent n=61; 2 segments under n=10 excluded	Quality here is decision correctness against labels, not a rubric score, across language fluency, region and tier. The status is the audit's own per-segment verdict: a gap wider than the target whose interval overlaps the best segment's is a gap this sample cannot resolve, so it is reported as not measured rather than as a failure. NFR-05 allows 15 points; this Framework target is 5. Full per-segment figures: fairness audit.
Repeat contacts	21.2% (history, n=80)	Halved	Not measured (NOT MEASURED)	An offline run cannot observe a customer returning	Baseline from history.repeat_contact: 17 of 80 of these tickets (21.2%) came back under human handling, so halved is 10.6% or lower. Leading indicator: 16 of 61 sent replies went to tickets the labels say must be held, the kind of reply that comes back as a repeat contact.
Availability	—	99.5%	100.0% (MET)	95% CI 95-100% (n=80)	80 of 80 tickets had every stage work. 0 hit a model-call or pipeline failure (degraded ticket, classifier or generator error, or a guardrail that could not run); 80 of 80 still received a decision, a failure escalating rather than erroring. One run's tickets are not uptime over time. At this n the interval cannot confirm 99.5%.

Table A.1: All twelve measures, graded run.

APPENDIX A**Full results table: local run**

From `evaluation/results/final_dev_local_50_20260927/results_table.md`. Run
harness-20260927T101501Z-32ab3564, 50 tickets, drafting llama3.1-8b-ctx8k, guardrails qwen2.5-7b-ctx8k,
threshold 0.85, cache off, hidden-set runs 0. Table A.2 lists every measure.

Measure	Baseline	Target	Achieved	Confidence	Notes
First contact resolution	42%	60%	60.0% correctly resolved (80.0% auto-sent) (MET)	95% CI 46-72% (n=50)	10 of 40 sent replies went to tickets the labels say must be held, so they are not counted as resolved: a wrong answer returns as a repeat contact. Correctness is agreement with labels.expected_route, not a check of reply quality.
Mean time to first reply	8 to 12 hrs	< 5 min	52.2 s (MET)	n=40 auto-sent tickets; median 51.1 s	Harness time from ticket read to reply ready; excludes queueing and delivery. The 10 escalated or blocked tickets get no automated reply (holding reply is out of scope, PRD Table 6), so their first reply comes from the human queue.
Satisfaction proxy	3.2 / 5	4.0	Not measured (NOT MEASURED)	No scored sample on this run	The Framework's method is human review of a stated sample of responses against a rubric, reporting sample size, rubric and scorers. Not done for this run. Nearest evidence: B-18, where one person scored 50 drafts from b21_openai_80_20260914 on the judge's RAG rubric (mean answer relevance 4.86/5) — a different rubric, a different run and a single scorer, so not a satisfaction proxy.
Escalation rate	58%	≤ 30%	20.0% (MET)	95% CI 11-33% (n=50)	6 escalated by routing and 4 blocked by a guardrail; 0 tickets were blocked because a check could not run (fail-safe), not because it found a fault.
Classification precision	—	85%	97.8% mean over the 18 classes with a precision denominator; 17 of 18 at ≥85% (NOT MET)	Precision n (predictions made per class): median 2; 16 classes were predicted fewer than 5 times and 0 never, so per-class figures are indicative only	Weakest: account_access 60% (n=5); api_key_issue 100% (n=2); api_usage_question 100% (n=3). Overall intent accuracy 96.0%. The target is per class, so it is met only if every class reaches 85%.
Hallucination rate	—	≤ 5%	Not measured to method (NOT MEASURED)	—	The Framework requires at least fifty responses assessed independently by two people, with their agreement reported. This project had one human assessor. Nearest evidence (B-18, drafts from b21_openai_80_20260914): one human found an unsupported claim in 1 of 50 drafts; the LLM judge in 5 of 50; the judge failed calibration (pooled Spearman 0.3492 against 0.70).
Citation accuracy	—	95%	Not measured to method (NOT MEASURED)	—	The Framework checks each citation against the sentence it is attached to. Sentence-level support was not assessed. Nearest evidence: the cited ARTICLE was one the labels expect in 88.9% of citations over 36 drafts (recall 84.7%) — a check of the right source, not of support.
Latency p95	—	< 3 s	77.9 s (NOT MET)	n=50 live tickets; max 88.0 s	End to end per ticket, including retrieval and every model call, run in sequence. Models: llama3.1-8b-ctx8k drafting, qwen2.5-7b-ctx8k guardrails.
Private data occurrences	—	0	0 (MET)	Automated scan of all 40 sent replies; manual sample review: not done	The scan uses the PII guardrail's own patterns (email, API key, phone, account number), independently of the judge's verdict. The guardrail blocked 0 drafts for private data before sending.

Measure	Baseline	Target	Achieved	Confidence	Notes
Cross-group variation	—	< 5 pts	30 pts (language_fluency: fluent vs non_fluent) (NOT MEASURED)	breach not significant: fluent n=40 against non_fluent n=10; 1 segments under n=10 excluded	Quality here is decision correctness against labels, not a rubric score, across language fluency, region and tier. The status is the audit's own per-segment verdict: a gap wider than the target whose interval overlaps the best segment's is a gap this sample cannot resolve, so it is reported as not measured rather than as a failure. NFR-05 allows 15 points; this Framework target is 5. Full per-segment figures: fairness audit.
Repeat contacts	24.0% (history, n=50)	Halved	Not measured (NOT MEASURED)	An offline run cannot observe a customer returning	Baseline from history.repeat_contact: 12 of 50 of these tickets (24.0%) came back under human handling, so halved is 12.0% or lower. Leading indicator: 10 of 40 sent replies went to tickets the labels say must be held, the kind of reply that comes back as a repeat contact.
Availability	—	99.5%	98.0% (NOT MET)	95% CI 90-100% (n=50)	49 of 50 tickets had every stage work. 1 hit a model-call or pipeline failure (degraded ticket, classifier or generator error, or a guardrail that could not run); 50 of 50 still received a decision, a failure escalating rather than erroring. One run's tickets are not uptime over time. At this n the interval cannot confirm 99.5%.

Table A.2: All twelve measures, local run.

APPENDIX B

Intent precision, recall and confusion matrix

Graded run. Precision is over the tickets predicted as the class; recall is over the tickets labelled as it. n/a means the class was never predicted (precision) or never labelled (recall). Table B.1 gives each class and Table B.2 the confusion matrix.

Intent	n labelled	n predicted	Precision	Recall
account_access	4	5	80.0%	100.0%
api_key_issue	6	6	100.0%	100.0%
api_usage_question	4	5	80.0%	100.0%
authentication_failure	3	2	100.0%	66.7%
billing_query	10	10	100.0%	100.0%
compliance_request	1	2	50.0%	100.0%
configuration_help	3	3	100.0%	100.0%
data_export	1	1	100.0%	100.0%
data_residency	6	5	100.0%	83.3%
database_issue	1	1	100.0%	100.0%
deployment_failure	5	5	100.0%	100.0%
feature_request	3	3	100.0%	100.0%
integration_help	3	3	100.0%	100.0%
onboarding	4	4	100.0%	100.0%
performance_degradation	3	3	100.0%	100.0%
quota_or_overage	2	2	100.0%	100.0%
rate_limit	1	0	n/a	0.0%
rollback_request	6	6	100.0%	100.0%
security_incident	4	4	100.0%	100.0%
sso_configuration	1	1	100.0%	100.0%
unclear_request	6	6	100.0%	100.0%
webhook_issue	3	3	100.0%	100.0%

Table B.1: Per-class figures.

Rows are the labelled intent and columns the predicted intent, numbered as the rows are. The diagonal holds correct classifications; blank cells are zero.

Labelled / predicted	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1. account_access	4																					
2. api_key_issue		6																				
3. api_usage_question			4																			
4. authentication_failure	1			2																		
5. billing_query					10																	
6. compliance_request						1																
7. configuration_help							3															
8. data_export								1														
9. data_residency						1			5													
10. database_issue										1												
11. deployment_failure											5											
12. feature_request												3										
13. integration_help													3									
14. onboarding														4								
15. performance_degradation															3							
16. quota_or_overage																2						
17. rate_limit			1																			
18. rollback_request																	6					
19. security_incident																		4				
20. sso_configuration																			1			
21. unclear_request																				6		
22. webhook_issue																					3	

Table B.2: Intent confusion matrix.

APPENDIX C

Fairness audit

Graded run, from `evaluation/results/final_val_paid_80_20260927/fairness/fairness_report.json`.

Decision correctness is agreement with the labelled route; resolution is over tickets that should be answered; unsafe sends are over tickets that should be held. Segments under n=10 are indicative. Table C.1 lists every segment.

Segment	n	Decision correct	Resolution	Unsafe sends	Send precision
language_fluency = fluent	61	80% (49/61)	92% (37/40)	43% (9/21)	80% (37/46)
language_fluency = non_fluent	19	63% (12/19)	100% (8/8)	64% (7/11)	53% (8/15)
customer_region = asia_pacific	21	76% (16/21)	93% (14/15)	67% (4/6)	78% (14/18)
customer_region = europe	25	76% (19/25)	94% (15/16)	56% (5/9)	75% (15/20)
customer_region = latin_america	7	86% (6/7)	100% (3/3)	25% (1/4)	75% (3/4)
customer_region = north_america	27	74% (20/27)	93% (13/14)	46% (6/13)	68% (13/19)
customer_tier = business	30	80% (24/30)	100% (21/21)	67% (6/9)	78% (21/27)
customer_tier = enterprise	8	100% (8/8)	100% (6/6)	0% (0/2)	100% (6/6)
customer_tier = standard	42	69% (29/42)	86% (18/21)	48% (10/21)	64% (18/28)
channel = chat	22	73% (16/22)	85% (11/13)	44% (4/9)	73% (11/15)
channel = docs_comment	16	81% (13/16)	100% (8/8)	38% (3/8)	73% (8/11)
channel = email	31	74% (23/31)	95% (20/21)	70% (7/10)	74% (20/27)
channel = forum	11	82% (9/11)	100% (6/6)	40% (2/5)	75% (6/8)
length = long	24	79% (19/24)	100% (15/15)	56% (5/9)	75% (15/20)
length = medium	28	71% (20/28)	89% (16/18)	60% (6/10)	73% (16/22)
length = short	28	79% (22/28)	93% (14/15)	38% (5/13)	74% (14/19)

Table C.1: Every segment.

APPENDIX D

Decision index

Table D.1 gives one line per decision.

ID	Decision
D-01	Model provider OpenRouter (Llama 3.1 8B): Setup Guide default, one API key.
D-01a	After the free model failed: paid diagnostics reported openly; the free-model gate closed by the local stack.
D-02	Embedding model all-MiniLM-L6-v2, verified on 357 development tickets.
D-02a	Cosine distance, threshold 0.25, set by measurement.
D-03	Decision log in SQLite, standard library only.
D-03a	Trace schema: one row per stage, keyed by ticket id and run id.
D-03b	A failed log write is recorded and the ticket escalated.
D-04	Chunking 800/120 with title and category prepended; hit@3 91.0 to 95.2%.
D-05, D-05a, D-05b	Confidence threshold 0.85 by sweep; the 0.95 precision floor is unmet.
D-06, D-06a	Router with a structural self-check; grounding as a blocking guardrail.
D-07	Four intents never auto-respond, plus unknown.
D-08	On-disk model reply cache.
D-09	Retrieval query is subject plus body, dense only.
D-10	No answerability gate from retrieval scores: AUC 0.64, held 0 of 14 wrong sends.
D-11	Private-data model detections must be value-shaped.
D-12	Guardrails concurrent, four workers, identical verdicts.
D-13	Caller-side deadline on every model call, 180 seconds.
D-14	Confidence-floor failure escalates at rule 6 instead of blocking.
D-15	Kill switch read on every routing call, tested.
D-16	Input-side injection check: escalate and record.
D-17	The harness works high-urgency tickets first by the classifier's rating; urgency accuracy is below a constant baseline on this data (7.2).

Table D.1: Decisions. Full text under `docs/adr/`.

APPENDIX E

Prompt register

From prompts/README.md. Every prompt names the requirements it serves, fences customer text as data, returns strict JSON and carries at least three test cases. Table E.1 lists them.

ID	Purpose	Version	Last changed
PR-CLASSIFY-01	Ticket intent + urgency + calibrated confidence	1.3	2026-09-26
PR-GENERATE-01	Grounded answer with citations	2.1	2026-09-23
PR-GUARDRAIL-PII-01	Detects PII in the drafted response (emails, keys, phones, account numbers, third-party names)	1.0	2026-09-03
PR-GUARDRAIL-GROUNDING-01	Verifies every factual claim in the drafted answer is supported by a cited passage	1.0	2026-09-03
PR-GUARDRAIL-TONESCOPE-01	Blocks replies that make commitments about refunds, delivery timings, or product roadmap items	1.0	2026-09-03
PR-GUARDRAIL-RELEVANCE-01	Checks the drafted reply addresses the question the customer actually asked	1.1	2026-09-20
PR-GUARDRAIL-DRAFT-01	One call returning the three draft-only guardrail verdicts: PII, tone/scope commitments, and answer relevance	1.0	2026-09-20
PR-EVAL-JUDGE-01	Scores one drafted reply on context relevance, groundedness and answer relevance (1-5 each)	1.1	2026-09-14

Table E.1: Prompts in the system and in evaluation.

APPENDIX F

Dataset profile

Development set, from each ticket's history block, which the Dataset Guide describes as the current baseline. Table F.1 gives the baseline by segment, Table F.2 the development set's composition and Table F.3 the two labelled sets side by side.

Segment	n	First-contact resolution	CSAT	Escalated	Median minutes to resolve
language_fluency = fluent	380	43.2%	2.95	56.8%	216
language_fluency = non_fluent	120	45.8%	3.04	54.2%	198
customer_tier = business	164	48.2%	2.91	51.8%	141
customer_tier = enterprise	83	37.3%	3.05	62.7%	369
customer_tier = standard	253	43.1%	2.98	56.9%	266

Table F.1: Human-handled baseline by segment.

Answerable from docs	Expected route	Tickets
no	escalate	143
yes	auto_respond	311
yes	escalate	46

Table F.2: Development set composition (n=500).

Set	Tickets	Answerable	Should auto-respond	History FCR	History repeat contact
development	500	71.4%	62.2%	43.8%	21.6%
validation	80	66.2%	60.0%	46.2%	21.2%

Table F.3: The two labelled sets compared.

APPENDIX G

Workbooks and artefacts

The filled workbooks and the other artefacts are submitted alongside this report; Table G.1 gives where each lives.

Artefact	Location
Stage 1 discovery workbook	workbooks/Stage_1_Discovery_Workbook.docx
Discovery notes and evidence tags	workbooks/discovery_notes.md
Stage 2 PRD	workbooks/Stage_2_PRD_Template.docx
Stage 3 prompt library	workbooks/Stage_3_Prompt_Library.docx, prompts/
Stage 4 sprint plan	workbooks/Stage_4_Sprint_Plan.docx
Stage 5 PRD revision log	workbooks/Stage_5_PRD_Revision_Log.docx
Effort log	workbooks/Effort_Log.docx
Architecture and ADRs	docs/architecture.md, docs/adr/
Evaluation Framework results	docs/Evaluation_Framework.docx
Governance workbook	docs/Governance_Framework.docx
Graded run	evaluation/results/final_val_paid_80_20260927/
Free-model run	evaluation/results/final_dev_local_50_20260927/

Table G.1: Where each submitted artefact lives.

APPENDIX H

Code listings

The routing precedence, from the docstring of `route._decide` in `src/route.py`, and the commands that produce every figure in Section 7 (Listing H.1 and Listing H.2).

```

"""The ordered rules. Returns (decision, trigger, reason).

Precedence, highest first:
0. kill switch – automatic answering halted by a human (D-15)
1. governance – an unlogged upstream decision (D-03b)
2. safety – a blocking guardrail did not pass
2b. injection – the ticket text tried to give instructions (R-03, D-16)
3. policy – intent never auto-answers (D-07)
4. coverage – retrieval found nothing
5. honesty – the generator declined to answer
6. quality – confidence below the floor
"""

```

Listing H.1: Routing precedence.

```

python -m evaluation.harness \
    --input data/validation_tickets.json \
    --output evaluation/results/final_val_paid_80_20260927
python -m evaluation.fairness_audit --results evaluation/results/final_val_paid_80_20260927

```

Listing H.2: The graded run and its fairness audit.

APPENDIX I

Declaration and AI-tool use

Declaration

All code, design decisions and evaluation figures in this submission are my own work, produced under the FDE Capstone brief between 26 August and 27 September 2026. Every requirement, ADR, prompt, test and result traces to the evidence it cites. No credentials appear in the source or the git history; tests/test_config_keys.py checks this in CI. The dataset is CloudServe's supplied fixture: 500 development tickets, 80 validation tickets, 29 articles, 200 reference responses and five interview transcripts. The hidden evaluation set was not accessed; runs against it: zero.

AI tools used

Table I.1 lists the AI tools used and how.

Tool	Where	How
Claude (Sonnet, Opus)	Prompts, ADRs, documentation prose, design options, this report's layout and evaluation analysis	As a pair-programming and drafting partner; every accepted output was reviewed and edited.
Editor completions	src/ in weeks two and three	Line-level completions inside my own drafts; no component generated end to end.
Models under test	The system itself	OpenRouter Llama 3.1 8B, then paid diagnostics, then gpt-4o-mini with gpt-4o for the graded run and local Ollama models for the free-model run.

Table I.1: AI tool use.

Third-party dependencies are pinned in requirements.txt. The harness, the six components, the guardrails, the decision log and every ADR were written for this project.